

# **Competitive Programming**

## **Section 14: Core Patterns Mixed Contest**

Competitive Programming Course

2026-08-02

# Table of contents

|           |                                                                     |           |
|-----------|---------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Why a mixed contest now</b>                                      | <b>3</b>  |
| <b>2</b>  | <b>Complete local contest set</b>                                   | <b>4</b>  |
| <b>3</b>  | <b>Focused stack and greedy ladder</b>                              | <b>5</b>  |
| <b>4</b>  | <b>Reinforcement ladder</b>                                         | <b>6</b>  |
| <b>5</b>  | <b>Greedy proof laboratory</b>                                      | <b>7</b>  |
| <b>6</b>  | <b>Level-up laboratory</b>                                          | <b>8</b>  |
| <b>7</b>  | <b>Phase 1 pattern map</b>                                          | <b>9</b>  |
| <b>8</b>  | <b>Contest triage</b>                                               | <b>10</b> |
| <b>9</b>  | <b>Keep the local reductions private</b>                            | <b>11</b> |
| <b>10</b> | <b>Complexity targets</b>                                           | <b>12</b> |
| <b>11</b> | <b>Common contest mistakes</b>                                      | <b>13</b> |
| 11.1      | Solving the first problem first automatically . . . . .             | 13        |
| 11.2      | Seeing a familiar word but missing a constraint . . . . .           | 13        |
| 11.3      | Forgetting endpoint conventions . . . . .                           | 13        |
| 11.4      | Printing sorted-query order . . . . .                               | 13        |
| <b>12</b> | <b>Debugging mixed sets</b>                                         | <b>14</b> |
| <b>13</b> | <b>Implementation discipline</b>                                    | <b>15</b> |
| <b>14</b> | <b>Submission checklist</b>                                         | <b>16</b> |
| <b>15</b> | <b>Exercises</b>                                                    | <b>17</b> |
| 15.1      | A through E . . . . .                                               | 17        |
| <b>16</b> | <b>How to find it: four guided investigations</b>                   | <b>18</b> |
| 16.1      | Investigation 1: smallest subsequence with several quotas . . . . . | 18        |
| 16.2      | Investigation 2: a wildcard walk through a gate . . . . .           | 20        |
| 16.3      | Investigation 3: selection under growing prefix budgets . . . . .   | 22        |
| 16.4      | Investigation 4: two sensors in every interval . . . . .            | 24        |
| <b>17</b> | <b>A reusable discovery card</b>                                    | <b>27</b> |

# 1 Why a mixed contest now

Sections 8 through 13 taught the core medium-level patterns:

- binary search on the answer;
- greedy exchange and constructive invariants;
- sweep line over events;
- monotonic stacks and dequeues;
- offline processing and coordinate compression.

Real contests do not label problems. This section trains the missing step:

Which pattern is this statement asking for?

The local set should be attempted like a short contest. Read all five problems first, estimate the intended complexity, and choose the order deliberately.

## 2 Complete local contest set

| Problem | Title                         |
|---------|-------------------------------|
| A       | Complete Window               |
| B       | Minimum Split Limit           |
| C       | Active Colors                 |
| D       | First Free Slot               |
| E       | Smallest Distinct Subsequence |

Problems D and E complete the phase coverage. In particular, E combines a greedy feasibility invariant with stack behavior, so one problem may exercise more than one category.

### 3 Focused stack and greedy ladder

After the A–E timed set, solve the following in order:

| Problem                              | New habit to practice                                          |
|--------------------------------------|----------------------------------------------------------------|
| F. Adjacent Cancellation             | use a stack as the reduced prefix                              |
| G. Next Smaller Distance             | keep unresolved indices monotonic                              |
| H. Smallest Fixed-Length Subsequence | undo a larger choice while deletion capacity remains           |
| I. Wildcard Parentheses              | maintain the interval of feasible balances                     |
| J. Interval Point Cover              | justify a locally extreme choice by exchange                   |
| K. Deadline Selection                | keep the best feasible subset by removing its costliest member |

F–H concentrate on stack invariants. I–K then move from a compact feasibility invariant to two exchange-based greedy algorithms. For each exercise, write the invariant before coding and compare against the brute-force random oracle afterward.

## 4 Reinforcement ladder

The next six problems revisit the points where the first ladder is most likely to feel abrupt:

| Problem                        | Connection to the first ladder                            |
|--------------------------------|-----------------------------------------------------------|
| L. Smallest After Deletions    | make H's deletion budget explicit                         |
| M. Required Letter Subsequence | add a second feasibility condition to H                   |
| N. Wildcard Choice Map         | combine I's balance intervals from both directions        |
| O. Minimum Parenthesis Depth   | place a monotone optimization around I's balance interval |
| P. Common Deadline             | isolate the shortest-job exchange used inside K           |
| Q. Event Attendance            | use a heap to choose the most urgent available interval   |

A productive order is  $H \rightarrow L \rightarrow M$ ,  $I \rightarrow N \rightarrow O$ , and  $P \rightarrow K \rightarrow Q$ . Before each implementation, state what makes a choice safe:

L and M: can the suffix still fill every required slot?  
N and O: which balances remain reachable after this prefix?  
P, K, and Q: which selected or available task is safest to keep?

L and P deliberately expose one invariant at a time. M, O, and Q then combine that invariant with an additional constraint. This is useful practice for recognizing a familiar greedy idea when the statement no longer looks identical.

## 5 Greedy proof laboratory

R–W form a second reinforcement set devoted to six different ways of justifying a greedy decision:

| Problem                             | Question that unlocks the greedy choice                              |
|-------------------------------------|----------------------------------------------------------------------|
| R. Coverage Patches                 | What is the first value that cannot currently be formed?             |
| S. Advantage Assignment             | Which available win consumes the least useful resource?              |
| T. Minimum Refueling                | When progress stops, which earlier opportunity extends it most?      |
| U. Circular Fuel Start              | After a failed segment, which starting candidates fail with it?      |
| V. Smallest Separated Rearrangement | Can the remaining multiset still be arranged after this character?   |
| W. Consecutive Grouping             | Which unfinished group becomes impossible if it is not extended now? |

These problems should not be approached by guessing a container. Write the answer to the table's question first. The resulting operation then suggests the implementation:

```
continuous representable range -> one endpoint
smallest useful match          -> sorted two pointers
best earlier opportunity       -> maximum heap
failed candidate block          -> one restart index
completion feasibility          -> frequency bounds
unfinished obligations           -> frequency maps
```

For each exercise, also write one plausible but incorrect local choice and try to break it with a small example. Greedy skill improves faster when the proof and the counterexample are practiced together.

## 6 Level-up laboratory

X–Z deliberately hide the useful state more deeply than the preceding problems:

| Problem                           | Discovery question                                             |
|-----------------------------------|----------------------------------------------------------------|
| X. Subarray Minimum Total         | How can all subarrays ending here be grouped by their minimum? |
| Y. Maximum Width Ramp             | Which indices can never be the best left endpoint?             |
| Z. Smallest Alternating Partition | What is forced when no existing group can accept the next bit? |

For X, do not begin by searching for both a previous-smaller and next-smaller formula. First list the minima of every suffix ending at the current position and observe how appending a smaller value changes them in blocks.

For Y, compare two possible left endpoints  $p < q$  with  $a[p] \leq a[q]$ . Endpoint  $q$  cannot produce a wider valid pair than  $p$ ; use that dominance relation to decide which indices deserve to remain.

For Z, separate the two objectives. First reason about the minimum number of groups. Only then ask which interchangeable group label makes the output lexicographically smallest. A minimum heap is for the second objective, not for proving the first.

Before reading the editorial, implement a distinct small oracle for each problem: enumerate subarrays for X, enumerate pairs for Y, and backtrack over canonical group assignments for Z.

## 7 Phase 1 pattern map

| Statement signal                                             | Likely pattern                 |
|--------------------------------------------------------------|--------------------------------|
| shortest/longest contiguous segment with maintainable counts | two pointers / sliding window  |
| minimize the maximum feasible value                          | binary search on answer        |
| intervals, active objects, point queries                     | sweep line                     |
| next greater/smaller, window maximum                         | stack or deque                 |
| queries can be sorted without changing meaning               | offline processing             |
| large values only need equality/order                        | coordinate compression or maps |
| local choice needs proof                                     | greedy exchange                |
| construct lexicographically smallest valid object            | greedy with suffix feasibility |

This table is not a replacement for proof. It is a triage tool.

## 8 Contest triage

Do a fast first pass:

```
for every problem:
    read the statement
    read constraints
    identify output type
    list one or two candidate patterns
    mark confidence: high / medium / low
```

Then solve in an order that maximizes reliable progress.

Questions to ask:

```
Is the answer a value or an object?
Are there many queries?
Can queries be reordered?
Does a pointer only move forward?
Is there a monotone yes/no predicate?
Do endpoints or ties matter?
```

## 9 Keep the local reductions private

The local problems in this section deliberately combine earlier patterns. Do not look for a one-word label or a prewritten template. Instead, decompose a statement into its independent requirements, then test whether one candidate model accounts for all of them:

```
objects that change  
query or objective being measured  
whether order is preserved or can be changed  
monotonicity or feasibility condition  
endpoint and tie conventions
```

Write that decomposition before implementing anything. The editorial contains the concrete reductions after you have made an attempt.

## 10 Complexity targets

For  $n, q \leq 200000$ , aim for:

```
O(n log n)
O((n+q) log(n+q))
O(n)
```

Avoid:

```
O(nq)
O(n^2)
iterating over coordinate ranges
```

For each local problem, derive a concrete target from the largest input dimension. A nested scan over two dimensions near 200000 is a warning sign; a logarithmic factor is often acceptable, but only after you can explain what operation causes it.

# 11 Common contest mistakes

## 11.1 Solving the first problem first automatically

First in statement order is not always easiest for you. Read the whole set.

## 11.2 Seeing a familiar word but missing a constraint

For example, a familiar word in a statement is not a proof that a familiar tool applies. State exactly what becomes easier or harder after one operation.

## 11.3 Forgetting endpoint conventions

For half-open intervals,  $[1, 5)$  is inactive at 5.

Write the endpoint convention as a one-line example before coding.

## 11.4 Printing sorted-query order

Any time you sort queries, attach original ids.

## 12 Debugging mixed sets

When a solution fails:

1. Is the chosen pattern valid?
2. Is the invariant true after every update?
3. Are boundary cases correct?
4. Is output order correct?
5. Can a brute-force oracle test small random cases?

For this section, write a tiny direct checker for the property requested by the statement. The optimized model and the tiny checker should be visibly different approaches.

## 13 Implementation discipline

Choose integer types from the largest possible accumulated value, make every index convention explicit, and keep original query indices whenever output order might differ from processing order. The needed container should follow from the model you wrote, not from the language you happen to prefer.

## 14 Submission checklist

- Did you identify the pattern before coding?
- Did you write the invariant or predicate in one sentence?
- Are all sums wide enough?
- Are interval endpoints handled with the correct tie order?
- Are query answers restored to original order?
- Did you test impossible and boundary cases?

# 15 Exercises

## 15.1 A through E

Attempt all five before opening the editorial. For each one, record the controlling constraint, one rejected approach, and the invariant or predicate your chosen model would need to maintain.

Then complete F through K as the focused ladder. Do not skip directly to H: F and G isolate the two stack invariants that H combines with a lexicographic greedy choice. Continue with L through Q for targeted reinforcement, use R through W to practice six different greedy proof shapes, and finish with X through Z as the level-up laboratory.

## 16 How to find it: four guided investigations

The goal of this chapter is not to memorize four more algorithms. It is to practice the reasoning that can produce an algorithm when the pattern is not yet obvious.

Use the same discovery loop for every unfamiliar problem:

1. Describe a correct brute-force choice or dynamic program.
2. Name precisely what makes it too slow.
3. Ask what information a future completion needs from the current prefix.
4. Look for structure in that information: monotonicity, an interval, an exchange, or one removable worst choice.
5. Write the invariant before choosing a data structure.
6. Prove one transition or greedy choice.
7. Compare the optimized algorithm with brute force on small instances.

The important step is usually step 3. A stack, heap, or pair of bounds is only the implementation of the information discovered there.

### 16.1 Investigation 1: smallest subsequence with several quotas

#### 16.1.1 New problem

Given a lowercase string `s`, choose a subsequence of exactly `k` characters. The answer must contain at least `need[a]` copies of `a` and at least `need[b]` copies of `b`. Find the lexicographically smallest valid answer. Assume a valid answer exists.

Example:

```
s = dcabacb
k = 4
need[a] = 1
need[b] = 2

answer = abab
```

This resembles a normal smallest fixed-length subsequence, but a locally small character can consume a position needed by a required letter.

#### 16.1.2 Step 1: write the brute-force decision

At every position, either take the character or skip it. A recursive state could contain:

```
current input position
number of answer positions already filled
number of selected a characters
number of selected b characters
```

This model is correct, but it has far too many states or explores exponentially many subsequences. Do not discard it yet: it tells us exactly which resources matter.

### 16.1.3 Step 2: understand the lexicographic objective

Two candidate answers are compared at their first different position. If the current stack ends with *d* and a new *c* arrives, replacing *d* by *c* is always lexicographically attractive.

The real question is not whether the replacement is attractive. It is:

```
After removing d, does at least one valid completion still exist?
```

This turns a vague greedy intuition into a precise feasibility test.

### 16.1.4 Step 3: list everything a pop can destroy

Popping the last selected character may cause two failures:

```
too few total characters remain to reach length k;
too few required letters remain to meet one of the quotas.
```

Maintain selected counts and suffix counts for every required letter. A pop is safe only if:

```
selected length after the pop + remaining input length >= k

and, for every required letter q:

selected[q] after the pop + remaining[q] >= need[q]
```

The suffix counts should include the current character during the pop test, because it is still available for the completion.

### 16.1.5 Step 4: reserve positions when pushing

Even if the stack has fewer than *k* characters, not every push is safe. After a hypothetical push, compute:

```
free positions = k - new stack length

missing quota =
    missing a copies + missing b copies
```

The push is allowed only when `free positions >= missing quota`. One future position cannot simultaneously provide both an `a` and a `b`, which is why the missing quantities are added.

For the example, once two of the four answer positions contain letters other than `a` and `b`, the other two positions cannot both satisfy one missing `a` and two missing `b` characters. The reservation check rejects that prefix before it becomes impossible.

### 16.1.6 Step 5: state the invariant

After processing each input position:

```
the stack is the smallest prefix obtainable by safe pops and pushes;  
its length is at most k;  
the unprocessed suffix can still fill all positions and all quotas.
```

The data structure is now obvious: only the last selected character is undone, so use a stack. The stack was a consequence of the replacement operation, not the starting point of the reasoning.

### 16.1.7 Why the greedy replacement is correct

When a larger last character is replaced by a smaller current character, the first changed answer position becomes smaller. The length and quota tests guarantee that some valid suffix still exists. Therefore the replacement makes the answer better without sacrificing feasibility.

If a pop is rejected, it would destroy at least one necessary resource. If a push is rejected, it would occupy a position reserved for a missing required letter. The algorithm therefore takes every safe lexicographic improvement and rejects only infeasible ones.

### 16.1.8 What transfers to other problems

For any problem asking for the lexicographically smallest valid object:

```
separate "is this choice smaller?" from "can the suffix still finish?"
```

The feasibility oracle might use suffix counts, a DP table, matching, or a remaining budget. The greedy construction becomes possible when that oracle is fast.

## 16.2 Investigation 2: a wildcard walk through a gate

### 16.2.1 New problem

A string contains `U`, `D`, and `?`. Starting at altitude zero, `U` increases the altitude by one, `D` decreases it by one, and `?` may be replaced by either move. The altitude may never be negative and must end at zero.

Additionally, immediately after move `p`, the walk must be at altitude `h`. Determine whether a valid replacement exists.

Example:

```
moves = ??U??  
p = 2  
h = 2
```

One valid replacement is UUDDDD.

### 16.2.2 Step 1: keep the complete reachable set

The direct dynamic program stores every altitude reachable after each prefix:

```
reachable[0] = {0}  
  
U: add 1 to every altitude  
D: subtract 1 and remove negative results  
?: take the union of both transitions
```

At position  $p$ , intersect the set with  $\{h\}$ . At the end, check whether zero is present.

This is already a correct solution. Its weakness is that a set may contain linearly many altitudes at each of linearly many positions.

### 16.2.3 Step 2: inspect small reachable sets

For question marks only:

```
prefix length 0: {0}  
prefix length 1: {1}  
prefix length 2: {0, 2}  
prefix length 3: {1, 3}  
prefix length 4: {0, 2, 4}
```

The values are not an ordinary interval: after an even number of moves only even altitudes are possible, and after an odd number only odd altitudes are possible. But they are a complete interval in steps of two.

This observation suggests storing only:

```
lowest reachable altitude  
highest reachable altitude  
the parity, which is determined by the prefix length
```

### 16.2.4 Step 3: prove there are no hidden gaps

Never compress a state set merely because the first few examples look dense. For  $?$ , an old altitude  $b$  creates  $b - 1$  and  $b + 1$ . The next old altitude is  $b + 2$ , and it creates  $b + 1$  and  $b + 3$ . The two results overlap at  $b + 1$ .

Therefore adjacent step-two intervals join without gaps. Fixed moves shift the whole set by one, and removing negative altitudes only cuts off its lower end. The compressed representation is exact by induction.

### 16.2.5 Step 4: express extra rules as intersections

The gate does not require a new algorithm. Immediately after move  $p$ , replace the current reachable set by its intersection with  $\{h\}$ :

```
if h is outside [low, high], impossible
if h has the wrong parity, impossible
otherwise set low = high = h
```

Then continue the same transitions.

For the example:

```
after first ?: {1}
after second ?: {0, 2}
gate at 2: {2}
after U: {3}
after ?: {2, 4}
after ?: {1, 3, 5}
after ?: {0, 2, 4, 6}
```

Zero is reachable at the end, so the answer is yes.

### 16.2.6 What transfers to other problems

Start from the full DP state set, then ask:

```
Does the set always have a provable shape?
Can a new restriction be represented as intersection with another set?
```

Possible shapes include an ordinary interval, a step-two interval, a prefix of sorted values, or a small number of disjoint intervals. If one operation can create a hole, the two-bound compression is invalid; keep more information.

This “write the large DP, inspect its states, then prove a compression” route is much more reliable than trying to guess `low` and `high` immediately.

## 16.3 Investigation 3: selection under growing prefix budgets

### 16.3.1 New problem

Items arrive in a fixed order. Item  $i$  has positive cost `cost[i]`. After the first  $i$  arrivals, the total cost of all selected items among that prefix must not exceed `budget[i]`. The budgets never decrease. Select as many items as possible.

Example:

```
cost    = 4, 2, 7, 3, 1
budget  = 4, 5, 8, 9, 10
```

The maximum number of selected items is three.

### 16.3.2 Step 1: identify when constraints become active

The constraint for prefix  $i$  cannot be checked before item  $i$  arrives. Once it has been checked, later decisions never change which items belong to that prefix; they can only remove previously selected items or add later items.

This suggests processing the constraints in their natural prefix order.

### 16.3.3 Step 2: tentatively accept the new opportunity

When an item arrives, selecting it is the only way the answer size might increase. Tentatively keep it.

If the total remains within the current budget, nothing needs repair. If the budget is exceeded, at least one selected item must be discarded.

### 16.3.4 Step 3: derive which item to discard

Every item contributes the same value to the objective: one selected item. Removing any one of them decreases the count by one. The only difference is how much capacity the removal recovers.

Removing the largest selected cost:

```
restores the most capacity now;  
leaves the smallest total cost for future constraints;  
does not reduce the selected count more than removing any other one item.
```

The phrase “remove the largest selected cost” tells us which container is needed: a maximum heap.

### 16.3.5 Step 4: trace the repair process

```
arrival 1: keep 4, total 4 <= budget 4  
arrival 2: total 6 > budget 5, remove 4, keep {2}  
arrival 3: total 9 > budget 8, remove 7, keep {2}  
arrival 4: keep 3, total 5 <= budget 9  
arrival 5: keep 1, total 6 <= budget 10
```

Three items remain.

One removal always suffices. Before adding the new item, the old total was at most the previous budget, which is at most the current budget. The largest selected cost is at least the newly added cost, so removing it returns the total to no more than its previous value.

### 16.3.6 Step 5: state and prove the invariant

After every prefix, the heap contains:

```
a maximum-cardinality feasible selection from that prefix;  
among those selections, one with minimum total cost.
```

If adding the new item is feasible, the cardinality increases and is clearly maximum. If it is infeasible, no solution can retain every currently selected item plus the new one, so the previous cardinality is the best possible. Removing the largest cost preserves that cardinality and produces the cheapest candidate obtainable by one removal. This preserves both parts of the invariant.

### 16.3.7 What transfers to other problems

Look for this pattern when:

```
constraints become active in a sortable order;  
you may tentatively accept every object;  
a violation can be repaired by deleting selected objects;  
all selected objects have equal objective value;  
one notion of "worst selected object" is justified by exchange.
```

Do not copy the heap rule when profits differ, when removed objects can return, or when repairing one constraint can damage an earlier non-nested constraint. Those changes invalidate the invariant and may require dynamic programming or a different greedy structure.

## 16.4 Investigation 4: two sensors in every interval

### 16.4.1 New problem

Each closed integer interval  $[1, r]$  describes a monitored zone, with  $1 < r$ . Choose the minimum number of distinct integer positions so that every interval contains at least two chosen positions.

Example:

```
[1,3], [3,5], [4,6]
```

One optimal set is  $\{2, 3, 5, 6\}$ .

### 16.4.2 Step 1: solve the urgent interval first

An interval with a small right endpoint loses its available positions sooner than an interval ending later. Sort by increasing right endpoint.

When the current interval is reached, count how many already selected points it contains:

```
two or more: add nothing
one:         add one point
zero:        add two points
```

The number of missing points is forced. The remaining question is where to put them.

### 16.4.3 Step 2: push forced choices as far right as possible

Choose the largest still-unselected integer positions inside the current interval. If two points are missing, these are usually  $r - 1$  and  $r$ ; if one of those is already selected, continue left until the required number of new positions has been added.

For the example:

```
[1,3] contains 0 selected points -> add 2 and 3
[3,5] contains only 3           -> add 5
[4,6] contains only 5           -> add 6
```

The final set is  $\{2,3,5,6\}$ .

### 16.4.4 Step 3: discover the exchange argument

Why choose points on the right rather than the left? Consider any optimal completion that satisfies the current interval using a newly added point  $p$  to the left of the greedy point  $g$ .

Replace  $p$  by  $g$ :

- the current interval still contains  $g$ ;
- every later interval has right endpoint at least the current right endpoint;
- if a later interval contained  $p$ , its left endpoint is at most  $p$ , so it also contains the larger point  $g$  as long as  $g$  is no farther than the current right endpoint.

Thus moving the forced new point right does not hurt the current or any later interval. Apply the exchange to every new point, preserving distinctness by using the rightmost unselected positions.

### 16.4.5 Step 4: generalize carefully

If every interval needs  $t$  points instead of two, the same reasoning suggests:

```
process intervals by increasing right endpoint;
measure the current coverage;
add the missing number of rightmost unselected points.
```

But changing closed intervals to half-open intervals changes the rightmost available integer. Allowing real-valued points can make “the next distinct position” ill-defined. The greedy idea transfers only after the domain and endpoint conventions are made precise.

### 16.4.6 What transfers to other problems

When a constraint forces a choice now, ask:

Among all choices satisfying the urgent constraint, which extreme choice preserves the largest set of options for constraints processed later?

For intervals sorted by end, that extreme is often the rightmost feasible choice. For other orderings or geometric objects, the correct extreme may be different, and the exchange proof must say exactly why.

## 17 A reusable discovery card

The four investigations can be summarized without referring to their stories:

| Situation                                    | First correct model     | Structure to search for                                 | Typical invariant                                                |
|----------------------------------------------|-------------------------|---------------------------------------------------------|------------------------------------------------------------------|
| lexicographically smallest valid object      | take/skip recursion     | fast suffix-feasibility oracle                          | chosen prefix is smallest among prefixes that remain completable |
| wildcard transitions                         | set of reachable states | interval, parity interval, or few components            | compressed set equals all reachable states                       |
| maximum count under nested capacities        | subset enumeration      | tentative acceptance plus removal of one worst item     | maximum count, then minimum resource use                         |
| minimum choices covering ordered constraints | hitting-set brute force | earliest urgent constraint plus extreme feasible choice | an optimum exists containing every greedy choice so far          |

Before coding a new problem, fill in these sentences:

```
My brute-force state or choice is _____.
It is too slow because _____.
The future only needs to know _____.
I believe that information has this structure: _____.
My invariant is _____.
The local choice is safe because _____.
The smallest counterexample to a wrong alternative is _____.
```

If the invariant sentence is still vague, implementation is premature. If the proof depends on “it seems better,” try to state an exchange or construct a small brute-force oracle. This habit is more reusable than any individual template in the section.